

Public Key Encryption

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



August 10, 2026

Table of Content

- 1 RSA Cryptosystem
- 2 RSA Encryption Scheme
- 3 RSA Digital Signature Scheme
- 4 Diffie–Hellman Key Exchange
- 5 ElGamal Encryption and Signature Scheme

RSA Cryptosystem

RSA Cryptosystem: An Introduction

RSA is a foundational public-key cryptosystem introduced in 1977 by **Rivest, Shamir, and Adleman**.

- **Encryption Scheme** Provides public-key encryption for establishing or transporting secret information.
- **Digital Signature:** Provides authentication and integrity using the same RSA key framework.

Underlying Assumption

Security is related to the computational difficulty of factoring

$$N = pq,$$

where p and q are large secret primes.

Historical Significance

RSA demonstrated that public-key cryptography could provide encryption and digital signatures without requiring a pre-shared secret key. It became one of the most widely deployed public-key cryptosystems.

Why Is Computing the RSA Private Exponent Difficult?

RSA publishes $N = pq, e$, while keeping p, q secret.
The private exponent is $d = e^{-1} \pmod{\phi(N)}$, $\phi(N) = (p-1)(q-1)$.

If $\phi(N)$ were known, computing d would be easy using the Extended Euclidean Algorithm:

$$ed \equiv 1 \pmod{\phi(N)}.$$

The difficulty is therefore obtaining $\phi(N)$ from the public value N .

For $N = pq$, knowing $\phi(N)$ immediately gives

$$\phi(N) = N - (p + q) + 1,$$

and hence

$$p + q = N - \phi(N) + 1.$$

Therefore p and q are the roots of

$$x^2 - (p + q)x + N = 0.$$

Conclusion

Computing $\phi(N)$ from N allows us to recover p and q . Thus, hiding the factorization hides the information needed to compute the RSA private exponent.

Extended Euclidean Algorithm

For integers a and b , the Extended Euclidean Algorithm computes

$$\gcd(a, b)$$

together with integers x, y satisfying

$$ax + by = \gcd(a, b).$$

Modular Inverse

If

$$\gcd(a, n) = 1,$$

then

$$ax + ny = 1.$$

Taking modulo n gives

$$x = a^{-1} \pmod{n}.$$

Extended Euclidean Algorithm: Concrete Algorithm

Given integers a and b , compute $\gcd(a, b)$ and integers x, y such that $ax + by = \gcd(a, b)$.

Initialize

$$r_0 = a, \quad r_1 = b, \quad s_0 = 1, \quad s_1 = 0, \quad t_0 = 0, \quad t_1 = 1.$$

While $r_1 \neq 0$, compute

$$q_i = \left\lfloor \frac{r_{i-1}}{r_i} \right\rfloor,$$

$$r_{i+1} = r_{i-1} - q_i r_i,$$

$$s_{i+1} = s_{i-1} - q_i s_i,$$

$$t_{i+1} = t_{i-1} - q_i t_i.$$

When $r_1 = 0$,

$$\gcd(a, b) = r_0$$

and

$$as_0 + bt_0 = r_0.$$

If $\gcd(a, b) = 1$, then

$$a^{-1} \equiv s_0 \pmod{b}.$$

Extended Euclidean Algorithm: Example

Find $3^{-1} \pmod{26}$.

Euclidean algorithm:

$$26 = 8(3) + 2$$

$$3 = 1(2) + 1$$

$$2 = 2(1) + 0.$$

Back-substitute:

$$\begin{aligned} 1 &= 3 - 2 \\ &= 3 - (26 - 8(3)) \\ &= 9(3) - 26. \end{aligned}$$

Thus,

$$9(3) + (-1)(26) = 1.$$

Taking modulo 26,

$$\boxed{3^{-1} \equiv 9 \pmod{26}}.$$

Verification:

$$3 \times 9 = 27 \equiv 1 \pmod{26}.$$

RSA Connection: For RSA, the same procedure computes $d = e^{-1} \pmod{\phi(N)}$.

RSA Encryption Scheme

It consists of the following algorithms.

- 1 **Key generation**
- 2 **Encryption** with public key
- 3 **Decryption** with secret key

RSA: Key Generation

RSA is based on the difficulty of factoring a large composite integer.

- 1 Choose two large random primes

$$p, q.$$

- 2 Compute

$$N = pq, \quad \phi(N) = (p-1)(q-1).$$

- 3 Choose e such that

$$1 < e < \phi(N), \quad \gcd(e, \phi(N)) = 1.$$

- 4 Compute the modular inverse

$$d = e^{-1} \pmod{\phi(N)}.$$

The keys are

$$pk = (N, e)$$

$$sk = (N, d)$$

RSA: Encryption and Decryption

Given a message representative $m \in \mathbb{Z}_N$:

Encryption

Anyone can encrypt using $pk = (N, e)$:

$$c = m^e \bmod N$$

Decryption

The owner uses $sk = (N, d)$:

$$m = c^d \bmod N$$

Correctness follows from

$$(m^e)^d = m^{ed} \equiv m \pmod{N}.$$

Practical RSA

Textbook RSA is deterministic and must not be used directly. Practical RSA encryption uses randomized padding such as RSA-OAEP.

RSA Digital Signature Scheme

It consists of the following algorithms.

- 1 **Key generation**
- 2 **Sign** with secret key
- 3 **Verify** with secret key

RSA Digital Signature: Key Generation

Choose two large primes

$$p, q.$$

Compute

$$N = pq, \quad \phi(N) = (p-1)(q-1).$$

Choose

$$e \text{ such that } \gcd(e, \phi(N)) = 1.$$

Compute

$$d = e^{-1} \pmod{\phi(N)}.$$

The keys are

$$\boxed{pk = (N, e)} \quad \boxed{sk = (N, d)}.$$

The public key is distributed, while d remains secret.

RSA Digital Signature: Signing

Given a message m , first compute its cryptographic hash

$$h = H(m).$$

The signer uses the secret exponent d to generate the signature:

$$\sigma = h^d \bmod N$$

Thus,

$$m \longrightarrow H(m) \longrightarrow \text{Sign}_{sk}(H(m)).$$

The signature σ is sent along with the message m .

Important

In practical RSA signatures, textbook RSA is replaced by a secure encoding scheme such as **RSA-PSS**.

RSA Digital Signature: Verification

Given the message m and signature σ , the verifier computes

$$h = H(m).$$

Using the public key (N, e) , compute

$$v = \sigma^e \bmod N.$$

Accept the signature iff

$$v = H(m)$$

Equivalently,

$$\text{Verify}_{pk}(m, \sigma) = 1$$

iff

$$\sigma^e \bmod N = H(m).$$

RSA Digital Signature: Correctness

During signing,

$$\sigma = H(m)^d \bmod N.$$

During verification,

$$\sigma^e \equiv (H(m)^d)^e = H(m)^{de} \pmod{N}.$$

Since

$$ed \equiv 1 \pmod{\phi(N)},$$

there exists an integer k such that

$$ed = 1 + k\phi(N).$$

Therefore, for $\gcd(H(m), N) = 1$,

$$H(m)^{ed} = H(m)^{1+k\phi(N)} \equiv H(m) \pmod{N}.$$

Hence,

$$\sigma^e \bmod N = H(m)$$

and the valid signature is accepted.

RSA: Classical Construction to Modern Practice

- **Core:** Modular exponentiation over

$$N = pq,$$

with security related to integer factorization.

- **Practical RSA:** Textbook RSA is insecure; use **RSA-OAEP** for encryption and **RSA-PSS** for signatures.
- **PKI:** RSA public keys can be certified by CAs and used for authentication and digital signatures.
- **Key Transport:** RSA was widely used to transport session keys, but modern protocols prefer ephemeral key agreement for forward secrecy.
- **Quantum Threat:** Shor's algorithm can efficiently factor large integers; RSA is therefore not post-quantum secure.
- **Transition:** RSA-based mechanisms are being replaced by post-quantum KEMs and signature schemes.

Perspective

RSA remains important in classical systems, but is not suitable for a post-quantum world.



Diffie–Hellman Key Exchange

Discrete Logarithm Problem and Diffie–Hellman Key Exchange

Discrete Logarithm Problem (DLP):

Given a cyclic group G , generator g , and

$$y = g^x,$$

it is easy to compute y from x , but computationally hard to recover x from (g, y) .

Diffie–Hellman Key Exchange

Public parameters: (G, q, g) , where G is a cyclic group of order q and g is a generator.

Alice chooses $a \xleftarrow{\$} \mathbb{Z}_q$, $A = g^a$.

Bob chooses $b \xleftarrow{\$} \mathbb{Z}_q$, $B = g^b$.

They exchange A and B and compute

$$K_A = B^a = g^{ab}, \quad K_B = A^b = g^{ab}.$$

Thus,

$$K_A = K_B = g^{ab}.$$

Security: An eavesdropper sees g^a and g^b but should not be able to compute g^{ab} under an appropriate DH hardness assumption.

ElGamal Encryption and Signature Scheme

ElGamal Encryption Scheme

Let (G, q, g) be public parameters, where G is a cyclic group of prime order q generated by g .

KeyGen

Choose

$$x \xleftarrow{\$} \mathbb{Z}_q.$$

Compute

$$y = g^x.$$

$$pk = y, \quad sk = x$$

Enc

For $m \in G$, choose fresh

$$r \xleftarrow{\$} \mathbb{Z}_q.$$

Compute

$$c_1 = g^r,$$

$$c_2 = m y^r.$$

$$c = (c_1, c_2)$$

Dec

Using $sk = x$, compute

$$m = c_2 (c_1^x)^{-1}.$$

Since

$$c_1^x = g^{rx} = y^r,$$

$$m = c_2 (c_1^x)^{-1}$$

Security

ElGamal security is based on the hardness of discrete-logarithm-related problems in the chosen group. Fresh r makes encryption probabilistic.

ElGamal-Style Digital Signature

Let $G = \langle g \rangle$ be a cyclic group of prime order q , and let H be a cryptographic hash function.

KeyGen

Choose

$$x \xleftarrow{\$} \mathbb{Z}_q^*.$$

Compute

$$y = g^x.$$

$$pk = y, \quad sk = x$$

Sign

Compute $h = H(m)$. Choose fresh

$$k \xleftarrow{\$} \mathbb{Z}_q^*.$$

Compute

$$r = g^k,$$
$$s = k^{-1}(h - xr) \pmod{q}.$$

$$\sigma = (r, s)$$

Verify

Compute

$$h = H(m).$$

Accept iff

$$g^h \equiv y^r r^s \pmod{q}$$

Security

Security is based on the discrete-logarithm-related hardness of the chosen group. The ephemeral k must be fresh and secret for every signature.

ElGamal-Style Signature: Correctness

During signing,

$$s = k^{-1}(H(m) - xr) \pmod{q}.$$

Therefore,

$$ks \equiv H(m) - xr \pmod{q},$$

and hence

$$xr + ks \equiv H(m) \pmod{q}.$$

During verification,

$$y^r r^s = (g^x)^r (g^k)^s = g^{xr+ks}.$$

Using

$$xr + ks \equiv H(m) \pmod{q},$$

we obtain

$$y^r r^s \equiv g^{H(m)} \pmod{q}.$$

Thus the verification equation

$$\boxed{g^{H(m)} \equiv y^r r^s \pmod{q}}$$

holds for every correctly generated signature.

Textbook:

- 1 Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. 4th Edition. Chapman & Hall/CRC Press, 2019. ISBN: 9780367148591.

Acknowledgment:

- The figures in this work were generated with the assistance of ChatGPT (OpenAI).



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.